

MLME Project

AI Usage Disclosure

18.07.2025

Group G2

Elisabeth Bünz

Paul Jollet

Rafael Kickartz

Gina Lüker

Ahmed Tantawy

1 Employed AI Models

- ChatGPT 4o
- GitHub Copilot
- DeepL

2 AI-assisted Code Portions and their Usage

The entire code was developed using GitHub Copilot. In particular, the docstrings were created using this feature. The “code suggestion” feature was especially usefull for plotting. This feature was also used when the same steps were performed for multiple datasets.

ChatGPT was primarily employed to generate scripts for Bayesian optimization. The first example in Chapter 3 shows the procedure of the iterative process involving ChatGPT in the Bayesian optimization of the hyperparameters of the NARX model.

DeepL was used to translate sections of the report, the final presentation and this disclosure of AI usage.

3 Example Usage

This chapter presents two examples of AI usage. The first example illustrates the iterative process involving ChatGPT in the Bayesian optimisation of NARX model hyperparameters. The second example demonstrates how ChatGPT and GitHub Copilot were employed for plotting.

Example 1

User:

<<ann.py>>

I've uploaded my `ann.py` file from the MLME project. Could you write a script that performs Bayesian Optimization for the hyperparameters using this model?

ChatGPT:

<<narx_bayesian_optimization.py>>

Sure! I've created a first version of the script. It uses `skopt` to optimize the lag and learning rate and evaluates them using your `train_model` function. You can find the file here:

`narx_bayesian_optimization.py`

User:

Thanks — the script basically works, but it doesn't fully follow Python coding conventions (PEP8). For example, split the code in functions and use main guard. Adding docstrings would improve readability and help people to understand the code.

ChatGPT:

<<narx_bayesian_optimization.py>>

You're absolutely right — thanks for the feedback! I've updated the script accordingly. It now features functions, main guard and docstrings. You can download the revised version here:

`narx_bayesian_optimization.py`

After downloading the code, it was reviewed and verified. The ranges for the hyperparameters were manually adjusted so that they made sense.

Example 2

User:

How can I plot the same data with a zoom on any interval?

```
# Plot
plt.figure(figsize=(12, 4))
plt.plot(y_true_test[:, i], 'o', label="True", alpha=0.5,
         markersize=1)
plt.plot(y_pred_test[:, i], '-', label="Pred", linewidth
         =0.5)
plt.fill_between(np.arange(len(lower)), lower, upper, color
                 ='gray', alpha=0.5, label=f"{int((1-alpha)*100)}% PI")
plt.title(f"CQR on test data: {target}")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(output_dir, f"cqr_interval_{target}
                        }.svg"))
plt.close()
```

ChatGPT:

To zoom in on a specific interval of your data, you can restrict the x-axis using `plt.xlim(start, end)` — where `start` and `end` are the indices of the desired range.

Here's how you can modify your existing code to include a zoom:

```
# Zoom on specific interval (e.g., from 0 to 100)
plt.figure(figsize=(12, 4))
    plt.plot(y_true_test[:, i], 'o', label="True",
             alpha=0.5, markersize=1)
    plt.plot(y_pred_test[:, i], '-', label="Pred",
             linewidth=0.5)
    plt.fill_between(np.arange(len(lower)), lower,
                     upper, color='gray', alpha=0.5,
                     label=f"{int((1-alpha)*100)}% PI")
    plt.title(f"CQR on test data (zoomed): {target}")
    plt.xlim(0, 100) # Adjust the x-axis limit for
zoom
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, f"
cqr_interval_zoomed_{target}.svg"))
    plt.close()
```

You can adjust the x-axis limit for the zoom if you want to analyze different time windows dynamically.

This code was also tested and refined manually.